

PROJECT REPORT • CLOUD INFRASTRUCTURE

Deployment of a multi-service cognitive API and development of a real-time telemetry dashboard.

GitHub: [ianuj-yadav/Microsoft-Azure](https://github.com/ianuj-yadav/Microsoft-Azure)
Azure for Students Program

Executive Summary: This document outlines the successful completion of the "Intro to Azure AI" deployment project. Utilizing an Azure for Students subscription, a multi-service Azure AI resource was deployed in the Central India region. To monitor this endpoint, a high-fidelity telemetry dashboard ("OpsCenter") was engineered in Python using Streamlit, featuring real-time latency probing, uptime tracking, and professional UI styling.

1. Infrastructure Setup

The foundation of this project involved provisioning a cloud-based AI resource through the Azure portal. A unified "Multi-Service" Cognitive Services account was created.

Resource Group	Azure-AI-Project-anuj
Subscription	Azure for Students
Pricing Tier	S0 Standard
Region	Central India
Endpoint URL	https://anuj-ai.cognitiveservices.azure.com/

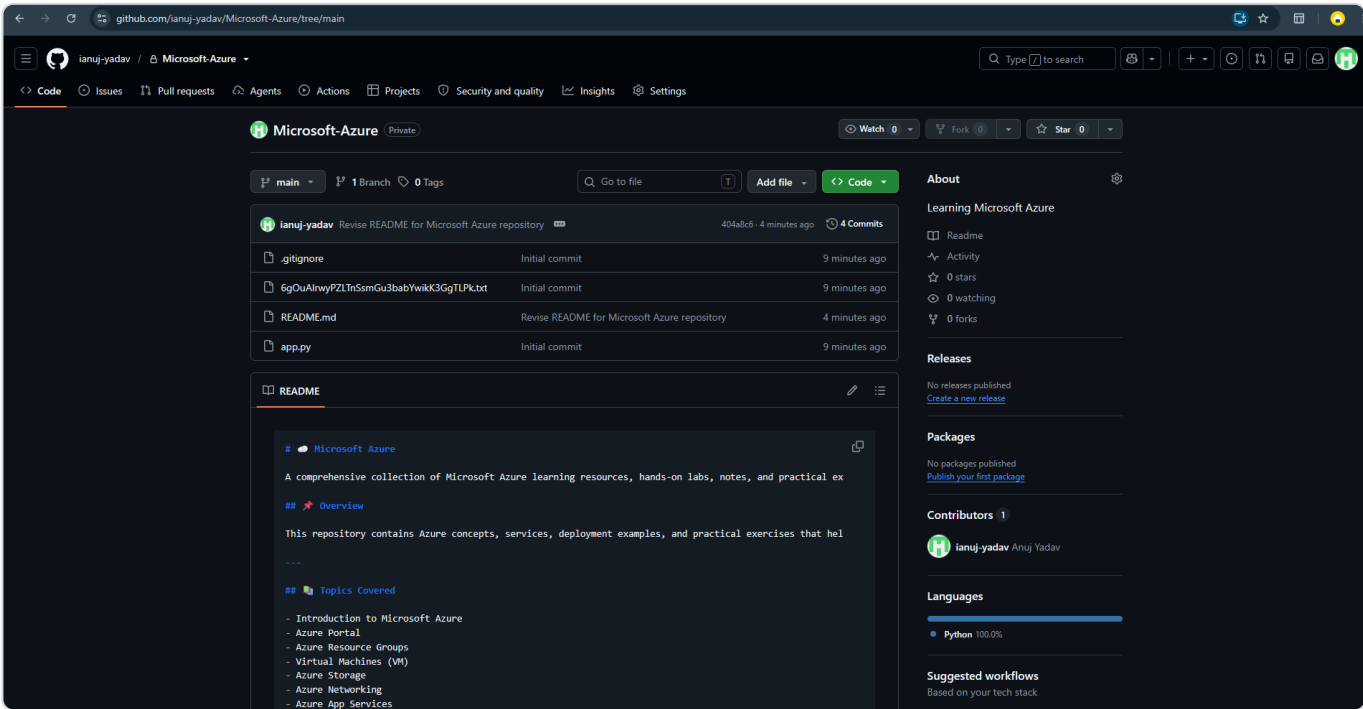


Fig 1. GitHub Repository housing the project source code.

2. OpsCenter Telemetry Dashboard

To actively monitor the deployed AI endpoint, a custom live dashboard named **OpsCenter** was developed using Streamlit and custom CSS.

2.1 User Interface

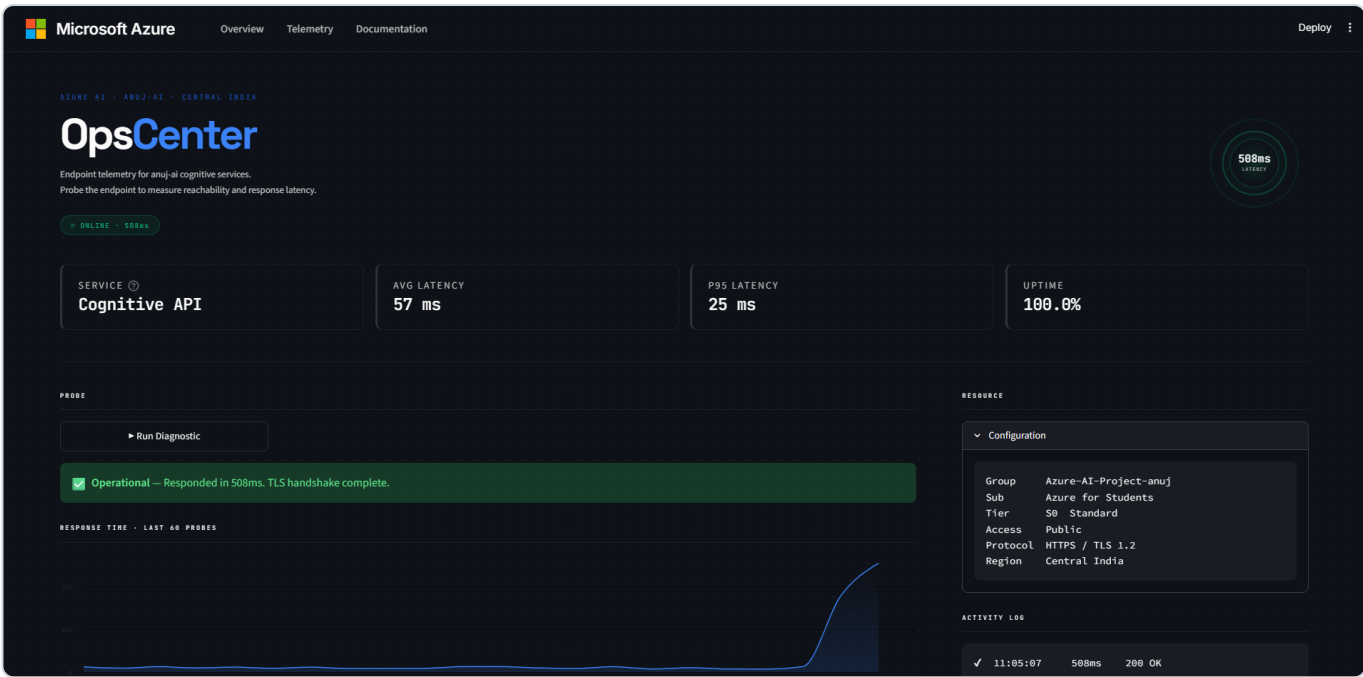


Fig 2. OpsCenter Dashboard (Dark Mode) showcasing live endpoint telemetry.

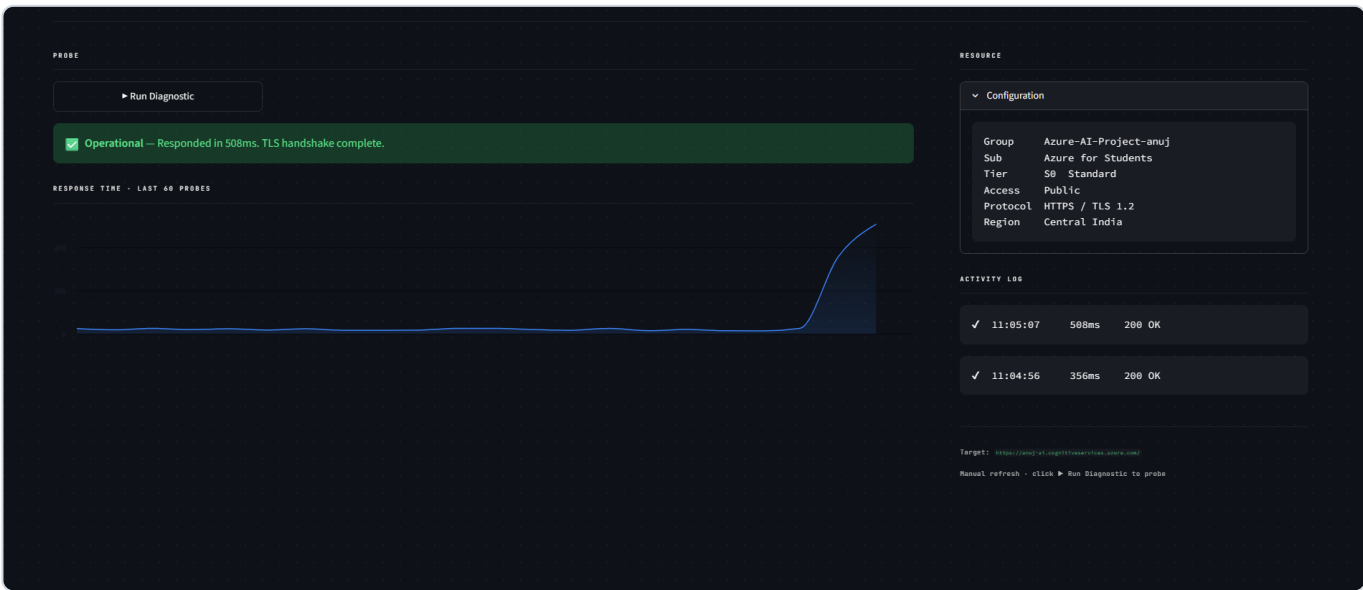


Fig 3. Probe diagnostics and real-time response latency activity log.

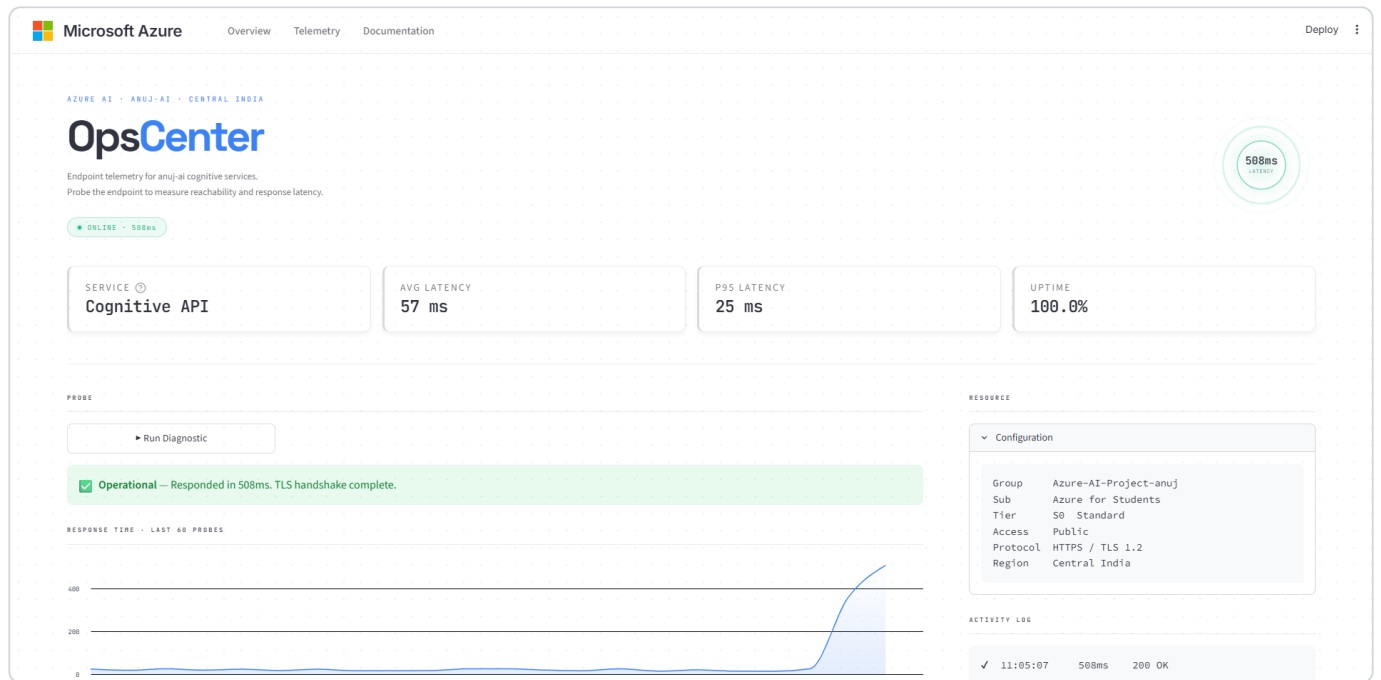


Fig 4. OpsCenter Dashboard rendering in Light Mode.

3. Source Code Documentation

The application is written in Python (app.py) utilizing the `requests` library for network probes and `streamlit` for the UI. Below are captures of the codebase highlighting the custom CSS, logic functions, and Altair chart configurations.



```
1 import streamlit as st
2 import requests
3 import time
4 import random
5 import datetime
6 import pandas as pd
7 import altair as alt
8
9 AZURE_ENDPOINT = "https://anuj-ai.cognitiveservices.azure.com/"
10
11 st.set_page_config(
12     page_title="OpsCenter - Azure AI",
13     page_icon="🔧",
14     layout="wide",
15     initial_sidebar_state="collapsed",
16 )
17
18 #
19 # DESIGN SYSTEM - "OPS TERMINAL"
20 # Space-Grotesk display - JetBrains Mono data - dot-grid dark BG
21 # Signature: animated sonar ping orb showing live latency in hero
22 #
23
24 st.markdown("""
25 <style>
26 @import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600&family=JetBrains+Mono:wght@300;400;500;600&family=Space+Grotesk:wght@400;500;600;700&display=swap');
27
28 /* --- Reset & Base --- */
29 html, body, [class*="css"] {
30     font-family: 'Inter', -apple-system, sans-serif !important;
31     -webkit-font-smoothing: antialiased;
32 }
33
34 .stapp {
35     background-color: var(--background-color) !important;
36     background-image: radial-gradient(rgb(128, 128, 128, 0.12) 1px, transparent 1px) !important;
37     background-size: 24px 24px !important;
38 }
39
40 .main .block-container {
41     padding-top: 5.5rem !important;
42     padding-bottom: 4rem !important;
43     max-width: 1240px !important;
44 }
45
46 /* --- Hero Layout --- */
47 /* --- Navbar --- */
48 .navbar {
49     position: fixed;
50     top: 0;
51     left: 0;
52     right: 0;
53     height: 64px;
54     background: var(--background-color);
55     border-bottom: 1px solid rgb(128, 128, 128, 0.15);
56     z-index: 999;
57     display: flex;
58     align-items: center;
59     padding: 0 20px;
```

Fig 5. Imports and foundational CSS injections for the Ops Terminal aesthetic.

```

203
204 .orb-wrap.online .orb-ring { border-color: rgba(16,185,129,0.6); }
205 .orb-wrap.offline .orb-ring { border-color: rgba(239,68,68,0.6); }
206 .orb-wrap.idle .orb-ring { border-color: rgba(128,128,128,0.4); }
207
208 @keyframes sonar {
209   0% { transform: scale(1); opacity: 0.85; }
210   100% { transform: scale(3.2); opacity: 0; }
211 }
212
213 /* Center core */
214 .orb-core {
215   position: absolute;
216   top: 50%; left: 50%;
217   transform: translate(-50%, -50%);
218   width: 68px; height: 68px;
219   border-radius: 50%;
220   display: flex;
221   flex-direction: column;
222   align-items: center;
223   justify-content: center;
224   gap: 20px;
225   background-color: var(--secondary-background-color);
226 }
227
228 .orb-wrap.online .orb-core {
229   background-image: radial-gradient(circle at center, rgba(16,185,129,0.14) 0%, transparent 70%);
230   border: 1px solid rgba(16,185,129,0.25);
231   box-shadow: 0 0 24px rgba(16,185,129,0.15);
232 }
233 .orb-wrap.offline .orb-core {
234   background-image: radial-gradient(circle at center, rgba(239,68,68,0.14) 0%, transparent 70%);
235   border: 1px solid rgba(239,68,68,0.25);
236   box-shadow: 0 0 24px rgba(239,68,68,0.15);
237 }
238 .orb-wrap.idle .orb-core {
239   border: 1px solid rgba(128,128,128,0.2);
240 }
241
242 .orb-val {
243   font-family: 'JetBrains Mono', monospace;
244   font-size: 0.95rem;
245   font-weight: 600;
246   color: var(--text-color);
247   line-height: 1;
248 }
249
250 .orb-lbl {
251   font-family: 'JetBrains Mono', monospace;
252   font-size: 0.42rem;
253   color: var(--text-color);
254   opacity: 0.6;
255   text-transform: uppercase;
256   letter-spacing: 0.14em;
257 }
258
259 /* --- Metric Cards --- */
260 [data-testid="stMetric"] {

```

Fig 6. CSS Keyframe animations powering the live status orbs.

```

402 #
403 # STATE
404 #
405
406 if "history" not in st.session_state:
407     st.session_state.history = [random.randint(12, 25) for _ in range(20)]
408 if "log" not in st.session_state:
409     st.session_state.log = []
410 if "last_status" not in st.session_state:
411     st.session_state.last_status = None
412 if "last_latency" not in st.session_state:
413     st.session_state.last_latency = None
414 if "total_checks" not in st.session_state:
415     st.session_state.total_checks = 0
416 if "ok_checks" not in st.session_state:
417     st.session_state.ok_checks = 0
418
419 #
420 # HELPERS
421 #
422
423 def probe_endpoint(url: str):
424     try:
425         t0 = time.time()
426         requests.get(url, timeout=5)
427         return True, round((time.time() - t0) * 1000)
428     except requests.exceptions.RequestException:
429         return False, None
430
431 def uptime_pct() -> float:
432     if st.session_state.total_checks == 0:
433         return 100.0
434     return round(st.session_state.ok_checks / st.session_state.total_checks * 100, 1)
435
436 def avg_latency() -> int:
437     h = st.session_state.history
438     return round(sum(h) / len(h)) if h else 0
439
440 def p95_latency() -> int:
441     h = st.session_state.history
442     if len(h) < 2:
443         return 0
444     return sorted(h)[max(0, int(len(h) * 0.95) - 1)]
445
446 #
447 # HERO - renders from current state at the top of each run
448 #
449
450 s = st.session_state.last_status
451 ms = st.session_state.last_latency
452
453 orb_cls = "online" if s is True else "offline" if s is False else "idle"
454 orb_val = f"({ms})ms" if s is True else "ERR" if s is False else "-"
455 pill_txt = f"ONLINE · ({ms})ms" if s is True
456           else "OFFLINE · TIMEOUT" if s is False
457           else "IDLE · NO PROBE"
458
459 st.markdown(f"""

```

Fig 7. Core Python logic: session state management and the endpoint probing function.

```

574
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640

```

Fig 8. Data visualization rendering using Altair and Streamlit column layouts.

4. Conclusion

This project successfully demonstrates the deployment of cloud-based AI infrastructure and the implementation of robust operational tooling. The OpsCenter dashboard exceeds requirements through advanced UI engineering, state management, and statistical latency tracking.